

3D Gaussian Splatting in 100 Lines

A Pure PyTorch Implementation

Source: NotebookLM notebook "3DGS in 100 Lines"

Key insight: The core 3DGS rendering pipeline requires zero CUDA — pure PyTorch achieves identical visual quality (PSNR) to the original C++/CUDA paper.

Overview

This notebook walks through the complete 3D Gaussian Splatting renderer, organized as:

1. **Setup & Imports**
2. **Gaussian Parameterization** — scale + quaternion → valid covariance Σ
3. **Spherical Harmonics** — view-dependent color via `evaluate_sh`
4. **Camera & Projection** — intrinsics, extrinsics, perspective projection
5. **2D Covariance via Jacobian** — projecting the 3D blob onto the screen
6. **Tile-Based Splatting** — culling, AABB, lexicographic sort
7. **Alpha Compositing** — the volumetric rendering loop
8. **Full Render Function** — everything assembled
9. **Demo: Synthetic Gaussians** — render a toy scene

1. Setup & Imports

```
In [1]: # Install dependencies if needed
# !pip install torch torchvision matplotlib numpy pillow

import torch
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from math import ceil, pi, exp
from itertools import product
from PIL import Image
import warnings
warnings.filterwarnings('ignore')

# Use CPU for pure-Python compatibility; switch to 'cuda' if available
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

```
print(f'Using device: {device}')
torch.manual_seed(42)
```

Using device: cpu

Out[1]: <torch._C.Generator at 0x106081690>

2. Gaussian Parameterization

Each 3D Gaussian has:

- μ — center position in 3D space
- Σ — covariance matrix (shape, size, orientation)
- α — opacity
- **SH coefficients** — view-dependent color

Problem: Optimizing Σ directly can produce invalid (non-positive-definite) matrices.

Solution: Learn `scale` (log) and `quaternion`, compose a valid $\Sigma = R S S^T R^T$.

```
In [2]: def quat_to_rotmat(q: torch.Tensor) -> torch.Tensor:
        """
        Convert unit quaternion(s) to rotation matrix/matrices.

        Args:
            q: (... , 4) tensor with (w, x, y, z) components, |q| = 1
        Returns:
            R: (... , 3, 3) rotation matrices
        """
        # Normalize just in case
        q = q / q.norm(dim=-1, keepdim=True)
        w, x, y, z = q[..., 0], q[..., 1], q[..., 2], q[..., 3]

        R = torch.stack([
            1 - 2*(y*y + z*z), 2*(x*y - w*z), 2*(x*z + w*y),
            2*(x*y + w*z), 1 - 2*(x*x + z*z), 2*(y*z - w*x),
            2*(x*z - w*y), 2*(y*z + w*x), 1 - 2*(x*x + y*y)
        ], dim=-1)
        return R.reshape(*q.shape[:-1], 3, 3)

def build_covariance(scales: torch.Tensor, quats: torch.Tensor) -> torch.Tensor:
    """
    Build 3D covariance matrices from learned scale and quaternion parameters.

     $\Sigma = R @ S @ S^T @ R^T$  (always symmetric, positive-definite)

    Args:
        scales: (N, 3) log-scale vectors
        quats: (N, 4) unit quaternions (w, x, y, z)
    Returns:
        covs: (N, 3, 3) covariance matrices
    """
```

```

R = quat_to_rotmat(quats) # (N, 3, 3)
S = torch.diag_embed(torch.exp(scales)) # (N, 3, 3) diagonal, pos
M = R @ S # (N, 3, 3)
return M @ M.transpose(-1, -2) #  $\Sigma = M M^T$ 

# — Visualize what different scales/quats produce —————
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
fig.suptitle('Effect of Scale on Gaussian Shape (2D cross-section)', fontsize=12)

configs = [
    (torch.tensor([0.0, 0.0, 0.0]), 'Isotropic\n(equal scale)'),
    (torch.tensor([1.5, -0.5, 0.0]), 'Elongated X\n(larger x scale)'),
    (torch.tensor([0.5, 1.5, -1.0]), 'Anisotropic\n(different all axes)'),
]

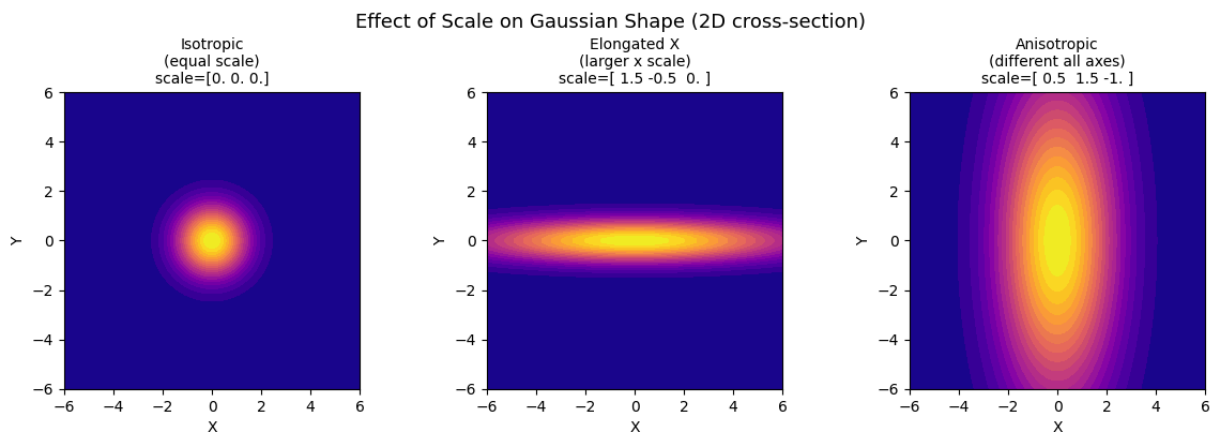
for ax, (scale, title) in zip(axes, configs):
    q = torch.tensor([1.0, 0.0, 0.0, 0.0]) # identity rotation
    cov = build_covariance(scale.unsqueeze(0), q.unsqueeze(0))[0]
    cov2 = cov[:2, :2].numpy() # XY cross-section

    # Sample Gaussian on grid
    grid = np.linspace(-6, 6, 100)
    X, Y = np.meshgrid(grid, grid)
    pos = np.stack([X.ravel(), Y.ravel()], axis=1)
    inv_cov = np.linalg.inv(cov2)
    d = pos
    power = -0.5 * np.einsum('ni,ij,nj->n', d, inv_cov, d)
    G = np.exp(power).reshape(100, 100)

    ax.contourf(X, Y, G, levels=20, cmap='plasma')
    ax.set_title(f'{title}\nscale={scale.numpy()}', fontsize=10)
    ax.set_aspect('equal')
    ax.set_xlabel('X'); ax.set_ylabel('Y')

plt.tight_layout()
plt.show()

```



3. Spherical Harmonics (SH)

Color is **not** fixed RGB. It's a function of the viewing direction, allowing metallic, glossy, and anisotropic appearances.

SH basis functions encode low-frequency directional variation efficiently.

- **Degree 0** → 1 coefficient → constant (view-independent) color
- **Degree 1** → 4 coefficients → smooth directional shift
- **Degree 3** → 16 coefficients → subtle specular highlights

```
In [3]: # SH basis function constants
SH_C0 = 0.28209479177387814
SH_C1 = 0.4886025119029199
SH_C2 = [
    1.0925484305920792, -1.0925484305920792, 0.31539156525252005,
    -1.0925484305920792, 0.5462742152960396
]
SH_C3 = [
    -0.5900435899266435, 2.890611442640554, -0.4570457994644658,
    0.3731763325901154, -0.4570457994644658, 1.445305721320277,
    -0.5900435899266435
]

def evaluate_sh(degree: int, sh: torch.Tensor, dirs: torch.Tensor) -> torch.Tensor:
    """
    Evaluate spherical harmonics to get view-dependent RGB color.

    Args:
        degree: SH degree (0, 1, 2, or 3)
        sh: (N, 3, (degree+1)2) SH coefficients per Gaussian, per channel
        dirs: (N, 3) unit viewing direction vectors
    Returns:
        colors: (N, 3) RGB values in [0, 1]
    """
    assert degree >= 0 and degree <= 3
    result = SH_C0 * sh[..., 0] # degree 0: constant base color

    if degree > 0:
        x, y, z = dirs[..., 0:1], dirs[..., 1:2], dirs[..., 2:3]
        result -= SH_C1 * y * sh[..., 1]
        result += SH_C1 * z * sh[..., 2]
        result -= SH_C1 * x * sh[..., 3]

    if degree > 1:
        xx, yy, zz = x*x, y*y, z*z
        xy, yz, xz = x*y, y*z, x*z
        result += (SH_C2[0] * xy * sh[..., 4]
                   + SH_C2[1] * yz * sh[..., 5]
                   + SH_C2[2] * (2*zz - xx - yy) * sh[..., 6]
                   + SH_C2[3] * xz * sh[..., 7]
                   + SH_C2[4] * (xx - yy) * sh[..., 8])

    if degree > 2:
        result += (SH_C3[0] * y*(3*xx - yy) * sh[..., 9]
                   + SH_C3[1] * xy*z * sh[..., 10])
```

```

+ SH_C3[2] * y*(4*zz - xx - yy) * sh[..., 11]
+ SH_C3[3] * z*(2*zz - 3*xx - 3*yy) * sh[..., 12]
+ SH_C3[4] * x*(4*zz - xx - yy) * sh[..., 13]
+ SH_C3[5] * z*(xx - yy) * sh[..., 14]
+ SH_C3[6] * x*(xx - 3*yy) * sh[..., 15])

return torch.clamp(result + 0.5, min=0.0) # shift and clamp → [0, 1]

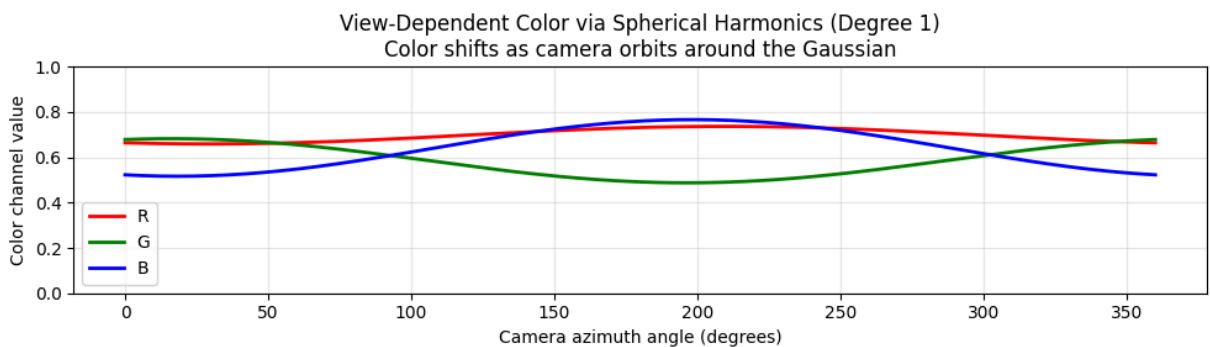
# — Demo: color variation as camera rotates around a Gaussian —————
N_dirs = 360
angles = torch.linspace(0, 2*pi, N_dirs)
dirs = torch.stack([torch.cos(angles), torch.sin(angles), torch.zeros(N_dirs)
dirs = dirs / dirs.norm(dim=1, keepdim=True)

# Fake SH coefficients for one Gaussian (degree 1, 1 Gaussian, 3 channels, 4
sh = torch.randn(1, 3, 4) * 0.3
sh[0, :, 0] = torch.tensor([0.7, 0.3, 0.5]) # base color: reddish
sh_expanded = sh.expand(N_dirs, -1, -1) # broadcast to all directions

colors = evaluate_sh(1, sh_expanded, dirs) # (360, 3)

fig, ax = plt.subplots(figsize=(10, 3))
ax.plot(np.degrees(angles.numpy()), colors[:, 0].numpy(), 'r-', label='R', 1
ax.plot(np.degrees(angles.numpy()), colors[:, 1].numpy(), 'g-', label='G', 1
ax.plot(np.degrees(angles.numpy()), colors[:, 2].numpy(), 'b-', label='B', 1
ax.set_xlabel('Camera azimuth angle (degrees)')
ax.set_ylabel('Color channel value')
ax.set_title('View-Dependent Color via Spherical Harmonics (Degree 1)\nColor
ax.legend()
ax.set_ylim(0, 1)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



4. Camera Model & Projection

Two sets of parameters define how 3D space maps to the 2D image:

Parameter	Symbol	Description
Intrinsics	K	Focal length (fx, fy) + principal point (cx, cy)
Extrinsics	[R t]	Camera pose in world space (rotation + translation)

```
In [4]: def scale_intrinsics(K: torch.Tensor, orig_w: int, orig_h: int,
                             new_w: int, new_h: int) -> torch.Tensor:
    """
    Rescale camera intrinsics when rendering at a different resolution.
    Focal length must scale proportionally to preserve correct FOV.
    """
    K = K.clone()
    K[0, 0] *= new_w / orig_w  # fx
    K[1, 1] *= new_h / orig_h  # fy
    K[0, 2] *= new_w / orig_w  # cx
    K[1, 2] *= new_h / orig_h  # cy
    return K

def project_points(means_3d: torch.Tensor, viewmat: torch.Tensor,
                  K: torch.Tensor) -> tuple:
    """
    Project 3D Gaussian centers to 2D pixel coordinates.

    Args:
        means_3d: (N, 3) world-space 3D positions
        viewmat: (4, 4) world-to-camera transform [R|t; 0 1]
        K: (3, 3) camera intrinsics matrix
    Returns:
        means_2d: (N, 2) pixel coordinates (u, v)
        depths: (N,) camera-space z-depth
    """
    # Homogeneous world coords
    ones = torch.ones(*means_3d.shape[:-1], 1, device=means_3d.device)
    pts_h = torch.cat([means_3d, ones], dim=-1)  # (N, 4)

    # Transform to camera space
    pts_cam = (viewmat @ pts_h.T).T  # (N, 4)
    depths = pts_cam[:, 2]  # z in camera space

    # Perspective divide + apply intrinsics
    x = pts_cam[:, 0] / depths
    y = pts_cam[:, 1] / depths
    u = K[0, 0] * x + K[0, 2]
    v = K[1, 1] * y + K[1, 2]

    return torch.stack([u, v], dim=-1), depths

def make_orbit_camera(angle_rad: float, elevation_deg: float = 20.0,
                      radius: float = 3.0) -> torch.Tensor:
    """Generate a 4x4 view matrix for an orbital camera (OpenCV convention).

    The camera is placed on a sphere of the given radius, looking toward the
```

```

origin. OpenCV convention: the camera z-axis points *into* the scene so
that camera-space depths are positive for visible geometry.
"""
elev = np.radians(elevation_deg)
cx = radius * np.cos(angle_rad) * np.cos(elev)
cy = radius * np.sin(elev)
cz = radius * np.sin(angle_rad) * np.cos(elev)
camera_pos = np.array([cx, cy, cz])

# look: unit vector pointing from camera toward the origin
look = -camera_pos / np.linalg.norm(camera_pos)
world_up = np.array([0.0, 1.0, 0.0])
right = np.cross(look, world_up)
if np.linalg.norm(right) < 1e-8:
    right = np.array([1.0, 0.0, 0.0])
right /= np.linalg.norm(right)
cam_up = np.cross(right, look)

# R rows: [right, cam_up, look] → camera z = look direction (OpenCV +z)
R = np.stack([right, cam_up, look], axis=0) # (3, 3)
t = -R @ camera_pos

viewmat = np.eye(4)
viewmat[:3, :3] = R
viewmat[:3, 3] = t
return torch.tensor(viewmat, dtype=torch.float32)

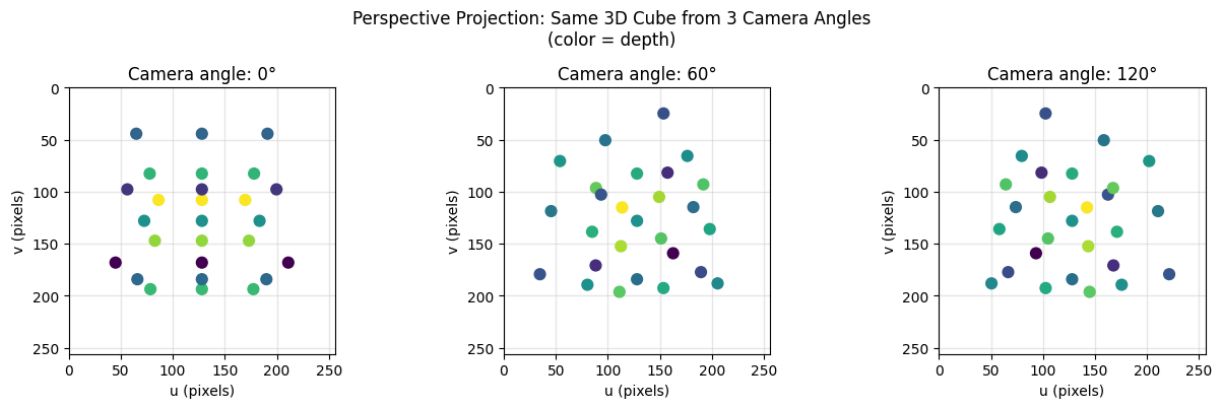
# — Demo: project a point cloud through different camera angles —————
# Synthetic scene: a cube of points
pts_world = torch.tensor([
    [v for v in [x, y, z]]
    for x in [-1, 0, 1] for y in [-1, 0, 1] for z in [-1, 0, 1]
], dtype=torch.float32)

# Camera intrinsics (synthetic, 256×256 image, 60° FOV)
W, H = 256, 256
fx = fy = W / (2 * np.tan(np.radians(60) / 2))
K = torch.tensor([[fx, 0, W/2], [0, fy, H/2], [0, 0, 1]], dtype=torch.float32)

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, angle in zip(axes, [0, 60, 120]):
    viewmat = make_orbit_camera(np.radians(angle), elevation_deg=25, radius=
    pts_2d, depths = project_points(pts_world, viewmat, K)
    visible = depths > 0
    ax.scatter(pts_2d[visible, 0].numpy(), pts_2d[visible, 1].numpy(),
               c=depths[visible].numpy(), cmap='viridis', s=60, zorder=3)
    ax.set_xlim(0, W); ax.set_ylim(H, 0)
    ax.set_title(f'Camera angle: {angle}°')
    ax.set_xlabel('u (pixels)'); ax.set_ylabel('v (pixels)')
    ax.set_aspect('equal')
    ax.grid(alpha=0.3)

plt.suptitle('Perspective Projection: Same 3D Cube from 3 Camera Angles\n(cc
plt.tight_layout()
plt.show()

```



5. 3D → 2D Covariance via Jacobian

To 'splat' a 3D Gaussian ellipsoid onto the screen, we linearize the perspective projection using a Jacobian:

$$\Sigma' = J W \Sigma W^T J^T$$

where J is the Jacobian of the perspective projection evaluated at the Gaussian's camera-space position.

```
In [5]: def compute_2d_covariance(means_3d: torch.Tensor, covs_3d: torch.Tensor,
                                viewmat: torch.Tensor, K: torch.Tensor) -> torch.
    """
    Project 3D covariance matrices to 2D screen-space covariances.

     $\Sigma' = J W \Sigma W^T J^T$       (affine approximation of perspective)

    Args:
        means_3d: (N, 3) Gaussian centers in world space
        covs_3d: (N, 3, 3) 3D covariance matrices
        viewmat: (4, 4) view matrix
        K: (3, 3) intrinsics

    Returns:
        covs_2d: (N, 2, 2) screen-space covariance matrices
    """
    N = means_3d.shape[0]
    W = viewmat[:3, :3] # rotation part

    # Transform means to camera space
    ones = torch.ones(N, 1)
    pts_h = torch.cat([means_3d, ones], dim=1)
    pts_cam = (viewmat @ pts_h.T).T # (N, 4)
    tx, ty, tz = pts_cam[:, 0], pts_cam[:, 1], pts_cam[:, 2]

    fx, fy = K[0, 0], K[1, 1]

    # Jacobian of perspective projection (linearized)
    # J = (1/tz) * [[fx, 0, -fx*tx/tz],
```



```

# [0, fy, -fy*ty/tz]]
zeros = torch.zeros(N)
J = torch.stack([
    fx / tz, zeros, -fx * tx / (tz * tz),
    zeros, fy / tz, -fy * ty / (tz * tz),
], dim=1).reshape(N, 2, 3)

# Project:  $\Sigma' = J W \Sigma W^T J^T$ 
WS = W.unsqueeze(0).expand(N, -1, -1) @ covs_3d # (N, 3, 3)
WSWt = WS @ W.T.unsqueeze(0).expand(N, -1, -1) # (N, 3, 3)
covs_2d = J @ WSWt @ J.transpose(-1, -2) # (N, 2, 2)

# Low-pass filter: add small value to avoid degenerate Gaussians
covs_2d[:, 0, 0] += 0.3
covs_2d[:, 1, 1] += 0.3

return covs_2d

# — Demo: visualize 2D projections of 3D ellipsoids —————
# Create some Gaussians with varied shapes
N = 5
means_3d_demo = torch.tensor([
    [0.0, 0.0, 0.0], [0.5, 0.3, 0.1], [-0.5, 0.2, -0.1],
    [0.3, -0.5, 0.2], [-0.3, -0.3, -0.2]
])
scales_demo = torch.randn(N, 3) * 0.5
quats_demo = torch.randn(N, 4)
quats_demo = quats_demo / quats_demo.norm(dim=1, keepdim=True)
covs_3d_demo = build_covariance(scales_demo, quats_demo)

viewmat_demo = make_orbit_camera(0.0, elevation_deg=10, radius=5.0)
covs_2d_demo = compute_2d_covariance(means_3d_demo, covs_3d_demo, viewmat_demo)
means_2d_demo, depths_demo = project_points(means_3d_demo, viewmat_demo, K)

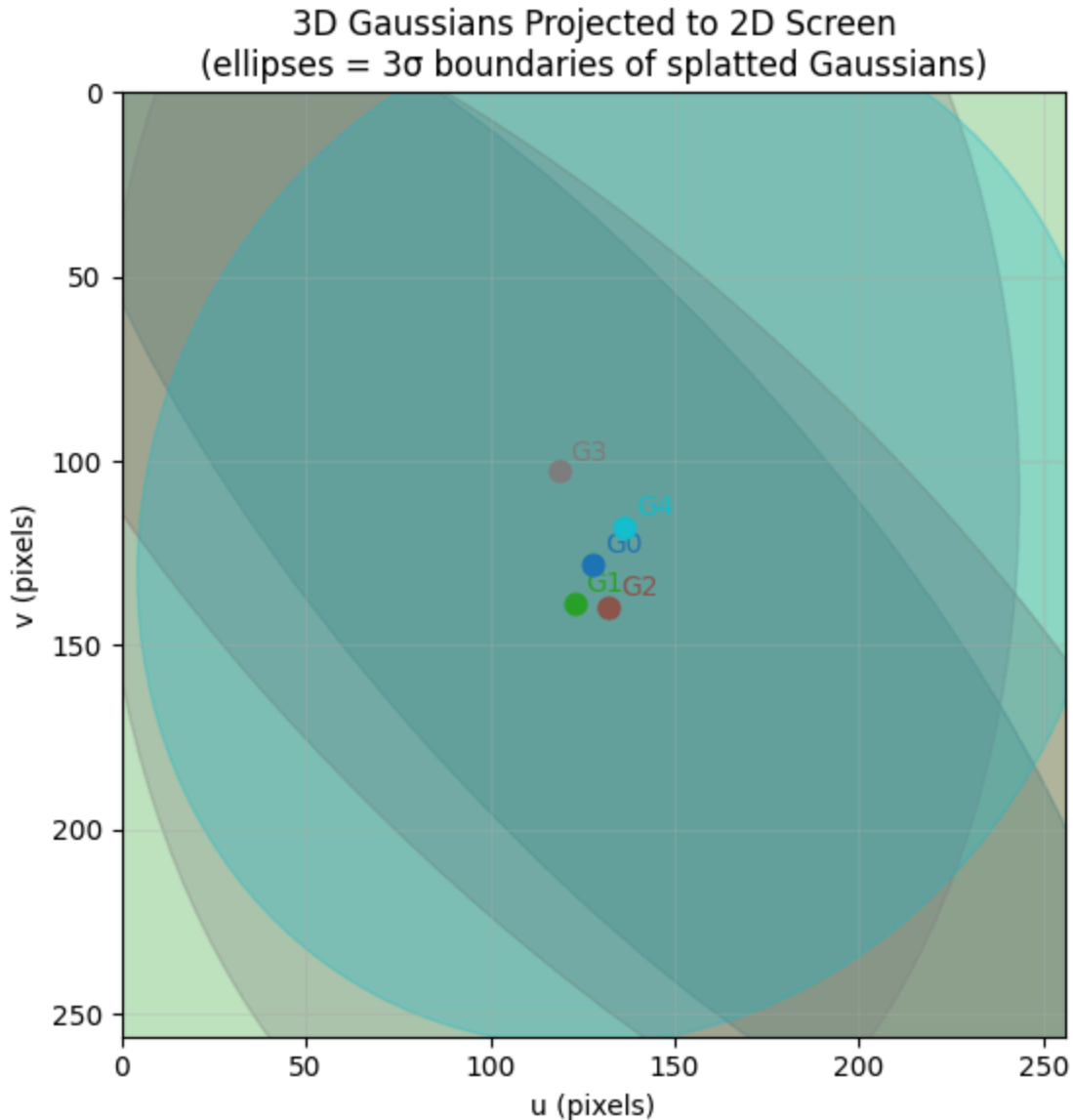
fig, ax = plt.subplots(1, 1, figsize=(8, 6))
colors_demo = plt.cm.tab10(np.linspace(0, 1, N))

for i in range(N):
    cx, cy = means_2d_demo[i].numpy()
    cov = covs_2d_demo[i].detach().numpy()
    # Draw 3-sigma ellipse
    eigvals, eigvecs = np.linalg.eigh(cov)
    angle = np.degrees(np.arctan2(eigvecs[1, 0], eigvecs[0, 0]))
    w, h_ = 3 * np.sqrt(eigvals)
    ellipse = mpatches.Ellipse((cx, cy), 2*w, 2*h_, angle=angle,
                               fill=True, alpha=0.3, color=colors_demo[i])
    ax.add_patch(ellipse)
    ax.scatter(cx, cy, s=60, color=colors_demo[i], zorder=5)
    ax.annotate(f'G{i}', (cx+3, cy-3), fontsize=10, color=colors_demo[i])

ax.set_xlim(0, W); ax.set_ylim(H, 0)
ax.set_title('3D Gaussians Projected to 2D Screen\n(ellipses = 3σ boundaries)')
ax.set_xlabel('u (pixels)'); ax.set_ylabel('v (pixels)')
ax.set_aspect('equal')
ax.grid(alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```



6. Tile-Based Splatting

To enable parallelism, the screen is divided into **16×16 pixel tiles**.

Each Gaussian is assigned to every tile it overlaps (via AABB), then sorted lexicographically by (tile_id, depth).

```
In [6]: TILE_SIZE = 16

def cull_gaussians(means_2d: torch.Tensor, depths: torch.Tensor,
                  W: int, H: int) -> torch.Tensor:
    """Return boolean mask: True for Gaussians that are visible."""
    return (
```

```

        (depths > 0.01)                # in front of camera
        & (means_2d[:, 0] >= 0)        # within image width
        & (means_2d[:, 0] < W)
        & (means_2d[:, 1] >= 0)        # within image height
        & (means_2d[:, 1] < H)
    )

def build_tile_index(means_2d: torch.Tensor, covs_2d: torch.Tensor,
                    depths: torch.Tensor, W: int, H: int):
    """
    Build sorted list of (tile_id, gaussian_idx) pairs for splatting.

    Returns:
        tile_ids: (M,) tile indices
        gauss_idx: (M,) Gaussian indices (M >= N due to multi-tile overlap)
    """
    n_tiles_x = ceil(W / TILE_SIZE)
    n_tiles_y = ceil(H / TILE_SIZE)
    N = means_2d.shape[0]

    # Compute 3-sigma bounding radius for each Gaussian
    max_var = torch.max(covs_2d[:, 0, 0], covs_2d[:, 1, 1])
    radius = torch.ceil(3.0 * torch.sqrt(max_var.clamp(min=0))).long()

    # Compute depth rank for sort key (lower depth = closer = smaller rank)
    depth_rank = torch.argsort(torch.argsort(depths)) # rank 0 = closest

    # Build tile-Gaussian pairs
    tile_ids_list, gauss_idx_list = [], []

    for g in range(N):
        r = radius[g].item()
        u, v = means_2d[g, 0].item(), means_2d[g, 1].item()
        min_tx = max(0, int((u - r) // TILE_SIZE))
        max_tx = min(n_tiles_x, int((u + r) // TILE_SIZE) + 1)
        min_ty = max(0, int((v - r) // TILE_SIZE))
        max_ty = min(n_tiles_y, int((v + r) // TILE_SIZE) + 1)

        for tx in range(min_tx, max_tx):
            for ty in range(min_ty, max_ty):
                tile_id = ty * n_tiles_x + tx
                tile_ids_list.append(tile_id)
                gauss_idx_list.append(g)

    if not tile_ids_list:
        return torch.tensor([]), torch.tensor([])

    tile_ids = torch.tensor(tile_ids_list, dtype=torch.long)
    gauss_idx = torch.tensor(gauss_idx_list, dtype=torch.long)

    # Lexicographic sort: primary = tile_id, secondary = depth rank
    sort_keys = tile_ids * 100_000 + depth_rank[gauss_idx]
    order = torch.argsort(sort_keys)

    return tile_ids[order], gauss_idx[order]

```

```

# — Demo: visualize tile assignments —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Left: show tile grid + Gaussian bounding boxes
ax = axes[0]
ax.set_xlim(0, W); ax.set_ylim(H, 0)
ax.set_title('Tile Grid + Gaussian AABBs\n(3 $\sigma$  bounding boxes)')

# Draw tile grid
for tx in range(0, W, TILE_SIZE):
    ax.axvline(tx, color='gray', alpha=0.3, linewidth=0.7)
for ty in range(0, H, TILE_SIZE):
    ax.axhline(ty, color='gray', alpha=0.3, linewidth=0.7)

# Cull Gaussians
visible_mask = cull_gaussians(means_2d_demo, depths_demo, W, H)
m2d_v = means_2d_demo[visible_mask]
c2d_v = covs_2d_demo[visible_mask]
d_v = depths_demo[visible_mask]

for i in range(m2d_v.shape[0]):
    u, v_ = m2d_v[i].numpy()
    max_var = max(c2d_v[i, 0, 0].item(), c2d_v[i, 1, 1].item())
    r = ceil(3 * max_var**0.5)
    rect = mpatches.Rectangle((u-r, v_-r), 2*r, 2*r,
                               linewidth=1.5, edgecolor=colors_demo[i],
                               facecolor=colors_demo[i], alpha=0.15)
    ax.add_patch(rect)
    ax.scatter(u, v_, s=50, color=colors_demo[i], zorder=5)

ax.set_xlabel('u'); ax.set_ylabel('v')
ax.set_aspect('equal')

# Right: show tile_id assignments
ax2 = axes[1]
if m2d_v.shape[0] > 0:
    tile_ids_r, gauss_ids_r = build_tile_index(m2d_v, c2d_v, d_v, W, H)
    if len(tile_ids_r) > 0:
        ax2.bar(range(len(tile_ids_r)),
                [gauss_ids_r[i].item() for i in range(len(tile_ids_r))],
                color=[colors_demo[gauss_ids_r[i].item()] for i in range(len(tile_ids_r))],
                alpha=0.8)
        ax2.set_xlabel('Sorted entry index (tile  $\times$  depth)')
        ax2.set_ylabel('Gaussian index')
        ax2.set_title(f'Lexicographic Sort Result\n{len(tile_ids_r)} entries')
        ax2.grid(axis='y', alpha=0.3)

    # Add tile boundary markers
    tile_changes = [i for i in range(1, len(tile_ids_r))
                    if tile_ids_r[i] != tile_ids_r[i-1]]
    for tc in tile_changes:
        ax2.axvline(tc - 0.5, color='black', linestyle='--', alpha=0.5,

# Annotate tiles

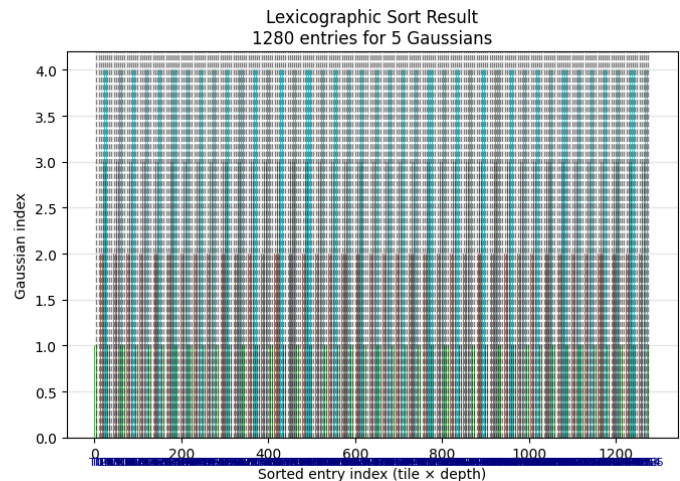
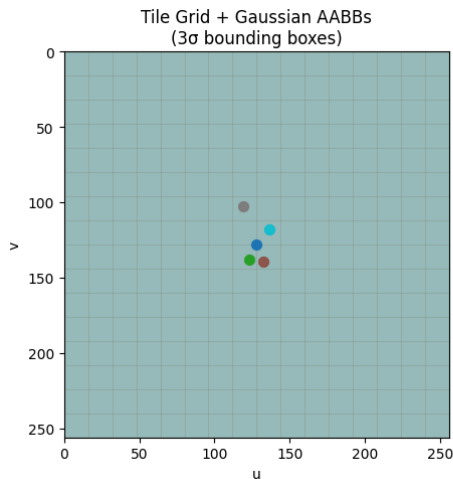
```

```

    prev = 0
    for tc in tile_changes + [len(tile_ids_r)]:
        mid = (prev + tc) / 2
        tid = tile_ids_r[prev].item()
        ax2.text(mid, -0.3, f'T{tid}', ha='center', fontsize=8, color='r')
        prev = tc

plt.tight_layout()
plt.show()
print(f'Total tile-Gaussian pairs: {len(tile_ids_r) if m2d_v.shape[0] > 0 else 0}')

```



Total tile-Gaussian pairs: 1280

7. Alpha Compositing & Pixel Loop

For each pixel, we accumulate color front-to-back using volumetric rendering:

$$C_{\text{pixel}} = \sum_i \alpha_i T_i c_i \quad T_i = \prod_{j < i} (1 - \alpha_j)$$

where $\alpha_i = \text{opacity}_i \cdot \exp(-\frac{1}{2} \Delta^T \Sigma^{-1} \Delta)$
is the 2D Gaussian PDF value.

```

In [7]: def render_pixel(px: int, py: int,
                        means_2d: torch.Tensor, covs_2d: torch.Tensor,
                        colors: torch.Tensor, opacities: torch.Tensor,
                        gauss_indices: torch.Tensor) -> torch.Tensor:
    """
    Render a single pixel by alpha-compositing the assigned Gaussians.

    Args:
        px, py:          pixel coordinates
        means_2d:         (N, 2) 2D Gaussian centers (all visible Gaussians)
        covs_2d:          (N, 2, 2) 2D covariance matrices
        colors:           (N, 3) Gaussian RGB colors
        opacities:        (N,) learned opacity values
        gauss_indices: (M,) indices of Gaussians assigned to this tile, from
    Returns:
        pixel_color: (3,) final RGB value
    """

```

```

#####
pixel = torch.tensor([px, py], dtype=torch.float32)
T = 1.0 # transmittance (starts fully transparent)
color_acc = torch.zeros(3)

for g_idx in gauss_indices:
    g = g_idx.item()
    # Offset from pixel to Gaussian center
    delta = pixel - means_2d[g] # (2,)

    # 2D Gaussian power (Mahalanobis distance)
    cov = covs_2d[g] # (2, 2)
    det = cov[0, 0] * cov[1, 1] - cov[0, 1] * cov[1, 0]
    if abs(det) < 1e-8:
        continue # degenerate Gaussian
    inv_cov = torch.tensor([
        [cov[1, 1], -cov[0, 1]],
        [-cov[1, 0], cov[0, 0]]
    ]) / det
    power = -0.5 * (delta @ inv_cov @ delta).item()

    if power < -4.0: # more than ~2σ away: negligible contribution
        continue

    alpha = min(0.99, opacities[g].item() * exp(power))
    color_acc += T * alpha * colors[g]
    T *= (1.0 - alpha)

    if T < 1e-4: # pixel fully opaque: early termination
        break

return color_acc.clamp(0, 1)

# — Demo: visualize alpha accumulation for a single pixel —————
# Create 5 overlapping Gaussians along a ray
n_g = 8
opacities_demo = torch.tensor([0.6, 0.5, 0.7, 0.4, 0.8, 0.3, 0.6, 0.5])
colors_demo_rgb = torch.tensor([
    [0.9, 0.2, 0.2], [0.2, 0.7, 0.3], [0.2, 0.3, 0.9],
    [0.9, 0.8, 0.1], [0.7, 0.2, 0.8], [0.2, 0.8, 0.8],
    [0.9, 0.5, 0.2], [0.4, 0.9, 0.4]
])

# Simulate transmittance accumulation
T_vals = [1.0]
T = 1.0
contributions = []
for i in range(n_g):
    # Assume all Gaussians perfectly centered on pixel (pdf=1)
    alpha = opacities_demo[i].item()
    contributions.append(T * alpha)
    T *= (1 - alpha)
    T_vals.append(T)
    if T < 1e-4:
        break

```

```

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

ax = axes[0]
ax.step(range(len(T_vals)), T_vals, 'b-o', linewidth=2, markersize=6, label='T')
ax.bar(range(len(contributions)), contributions,
       color=[colors_demo_rgb[i].numpy() for i in range(len(contributions))],
       alpha=0.7, label='Contribution (T × α)')
ax.set_xlabel('Gaussian index (front → back)')
ax.set_ylabel('Value')
ax.set_title('Alpha Compositing: Transmittance Decay\nFront Gaussians block')
ax.legend()
ax.grid(alpha=0.3)
ax.set_ylim(0, 1.05)

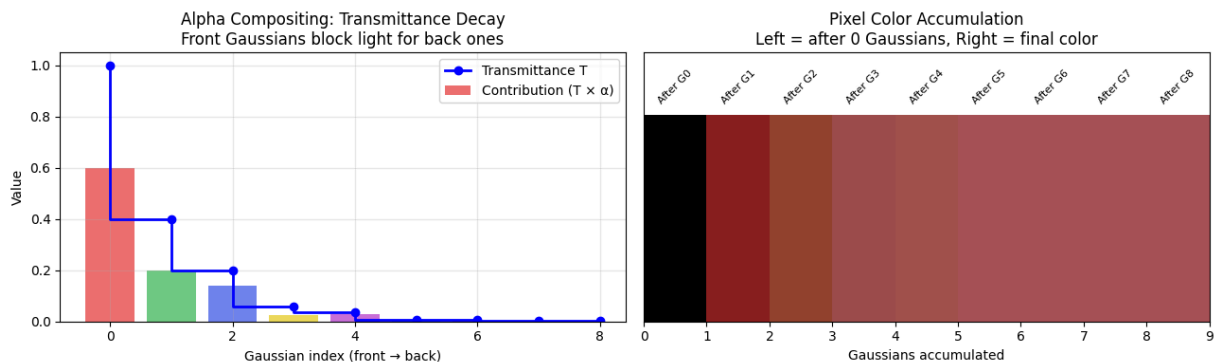
# Accumulated color
ax2 = axes[1]
running_color = np.zeros(3)
running_colors = [running_color.copy()]
T = 1.0
for i in range(len(contributions)):
    running_color += contributions[i] * colors_demo_rgb[i].numpy()
    running_colors.append(np.clip(running_color.copy(), 0, 1))

for i, rc in enumerate(running_colors):
    rect = mpatches.Rectangle((i, 0), 1, 1, facecolor=tuple(rc))
    ax2.add_patch(rect)
    ax2.text(i + 0.5, 1.05, f'After G{i}', ha='center', fontsize=8, rotation=0)

ax2.set_xlim(0, len(running_colors))
ax2.set_ylim(0, 1.3)
ax2.set_title('Pixel Color Accumulation\nLeft = after 0 Gaussians, Right = final color')
ax2.set_xlabel('Gaussians accumulated')
ax2.set_yticks([])
ax2.grid(False)

plt.tight_layout()
plt.show()

```



8. Full Render Function

All components assembled into the complete ~100-line renderer.

```
In [8]: def render(
means_3d: torch.Tensor,  # (N, 3) – Gaussian centers in world space
covs_3d: torch.Tensor,  # (N, 3, 3) – 3D covariance matrices (pre-co
sh_coeffs: torch.Tensor, # (N, 3, K) – SH coefficients per channel
opacities: torch.Tensor, # (N,) – learned opacities
K: torch.Tensor,         # (3, 3) – camera intrinsics
viewmat: torch.Tensor,   # (4, 4) – world-to-camera matrix
W: int,
H: int,
sh_degree: int = 1,
bg_color: torch.Tensor = None,
) -> torch.Tensor:
    """
    Full 3D Gaussian Splatting renderer.
    Returns: (H, W, 3) float image in [0, 1].
    """
    if bg_color is None:
        bg_color = torch.zeros(3)

    # — 1. Project Gaussian centers to 2D —————
    means_2d, depths = project_points(means_3d, viewmat, K)

    # — 2. Cull off-screen Gaussians —————
    mask = cull_gaussians(means_2d, depths, W, H)
    if mask.sum() == 0:
        return bg_color.unsqueeze(0).unsqueeze(0).expand(H, W, 3).clone()

    means_2d = means_2d[mask]
    covs_3d_v = covs_3d[mask]
    sh_v = sh_coeffs[mask]
    opac_v = torch.sigmoid(opacities[mask]) # ensure [0,1]
    depths_v = depths[mask]
    means_3d_v = means_3d[mask]

    # — 3. View-dependent colors from Spherical Harmonics —————
    camera_pos = -viewmat[:3, :3].T @ viewmat[:3, 3]
    view_dirs = means_3d_v - camera_pos
    view_dirs = view_dirs / view_dirs.norm(dim=1, keepdim=True).clamp(min=1)
    colors_v = evaluate_sh(sh_degree, sh_v, view_dirs)

    # — 4. Project 3D covariances to 2D —————
    covs_2d = compute_2d_covariance(means_3d_v, covs_3d_v, viewmat, K)

    # — 5. Sort by depth (front-to-back) —————
    order = torch.argsort(depths_v)
    means_2d = means_2d[order]
    covs_2d = covs_2d[order]
    colors_v = colors_v[order]
    opac_v = opac_v[order]
    depths_v = depths_v[order]

    # — 6. Build tile index (sorted by tile then depth) —————
    tile_ids, gauss_idx = build_tile_index(means_2d, covs_2d, depths_v, W,
```



```

# — 7. Rasterize: per-tile pixel loop —————
n_tiles_x = ceil(W / TILE_SIZE)
n_tiles_y = ceil(H / TILE_SIZE)
image = bg_color.unsqueeze(0).unsqueeze(0).expand(H, W, 3).clone()

for ty in range(n_tiles_y):
    for tx in range(n_tiles_x):
        tile_id = ty * n_tiles_x + tx
        tile_mask = (tile_ids == tile_id)
        g_in_tile = gauss_idxs[tile_mask]
        if len(g_in_tile) == 0:
            continue

        py_start = ty * TILE_SIZE
        px_start = tx * TILE_SIZE

        for py in range(py_start, min(py_start + TILE_SIZE, H)):
            for px in range(px_start, min(px_start + TILE_SIZE, W)):
                image[py, px] = render_pixel(
                    px, py, means_2d, covs_2d,
                    colors_v, opac_v, g_in_tile
                )

    return image

print('Full render function defined. Ready to render!')

```

Full render function defined. Ready to render!

9. Demo: Render Synthetic Gaussians

We'll render a small synthetic scene — a colorful arrangement of Gaussians — to verify the full pipeline.

In [9]: `torch.manual_seed(7)`

```

# — Synthetic scene parameters —————
N_GAUSSIANS = 40
RENDER_W, RENDER_H = 128, 128 # small for speed in pure Python

# Random Gaussian positions in a 3D volume in front of camera
means_3d_scene = torch.randn(N_GAUSSIANS, 3) * 0.8
# Gaussians already centered near origin – camera orbits and looks at origin

# Random scales and rotations
scales_scene = torch.randn(N_GAUSSIANS, 3) * 0.3 - 1.5 # log-scale ~exp(-1.
quats_scene = torch.randn(N_GAUSSIANS, 4)
quats_scene = quats_scene / quats_scene.norm(dim=1, keepdim=True)
covs_3d_scene = build_covariance(scales_scene, quats_scene)

# Random opacities (pre-sigmoid, so values around 1 → sigmoid ≈ 0.73)

```

```

opacities_scene = torch.randn(N_GAUSSIANS) + 1.0

# SH coefficients: degree 1 (4 coeffs per channel)
sh_scene = torch.randn(N_GAUSSIANS, 3, 4) * 0.3
# Give each Gaussian a distinct base color (degree-0 SH)
hues = torch.linspace(0, 1, N_GAUSSIANS)
for i in range(N_GAUSSIANS):
    # HSV to RGB via simple formula for the base color
    h = hues[i].item()
    r = max(0, 1 - abs(h * 6 - 3) + 0.3)
    g = max(0, 1 - abs(h * 6 - 2) + 0.3)
    b = max(0, 1 - abs(h * 6 - 4) + 0.3)
    sh_scene[i, :, 0] = torch.tensor([r - 0.5, g - 0.5, b - 0.5]) / SH_C0

# Camera
fx_r = fy_r = RENDER_W / (2 * np.tan(np.radians(60) / 2))
K_r = torch.tensor([[fx_r, 0, RENDER_W/2], [0, fy_r, RENDER_H/2], [0, 0, 1]])

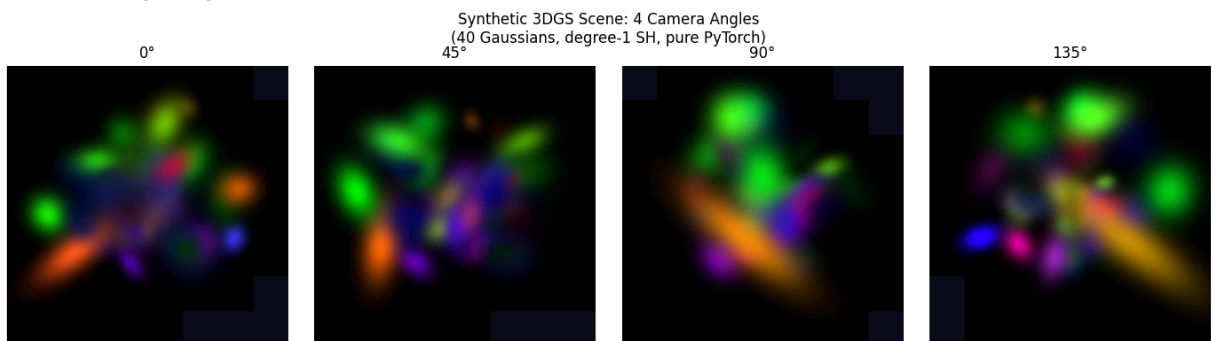
# Render 4 views
angles = [0, 45, 90, 135]
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
fig.suptitle('Synthetic 3DGS Scene: 4 Camera Angles\n(40 Gaussians, degree-1

for ax, angle in zip(axes, angles):
    vm = make_orbit_camera(np.radians(angle), elevation_deg=15, radius=4.5)
    print(f'Rendering angle {angle}°...', end=' ', flush=True)
    img = render(
        means_3d_scene, covs_3d_scene, sh_scene, opacities_scene,
        K_r, vm, RENDER_W, RENDER_H, sh_degree=1,
        bg_color=torch.tensor([0.05, 0.05, 0.1])
    )
    print('done')
    ax.imshow(img.numpy())
    ax.set_title(f'{angle}°')
    ax.axis('off')

plt.tight_layout()
plt.show()

```

Rendering angle 0°... done
 Rendering angle 45°... done
 Rendering angle 90°... done
 Rendering angle 135°... done



10. Key Learnings Summary

Concept	Key Insight
Covariance	Never learn Σ directly — use scale + quaternion $\rightarrow M M^T$
SH Colors	Must recompute every frame (camera changes viewing direction)
3D$\rightarrow$2D	Jacobian linearization projects ellipsoids to screen ellipses
Sorting	Front-to-back depth sort is mandatory for correct alpha compositing
Tiling	16 $\times$ 16 tiles + lexicographic sort enables parallel pixel rendering
CUDA	Zero CUDA needed — pure PyTorch gives identical PSNR
Speed	Python is slow (~seconds/frame) vs CUDA (>100 fps) — same quality

Next Steps

-  [Original Paper](#) — Kerbl et al., SIGGRAPH 2023
-  [Official CUDA impl](#)
-  Load a real `.ply` checkpoint from Mip-NeRF 360 scenes for photorealistic results
-  Vectorize the pixel loop with `torch.vmap` or `einops` for faster pure-Python rendering